

SIG731 : Task 1P

Introduction to Python and Jupyter Notebooks

1 Introduction

This task is related to Module 1; see the *Learning Resources* on the unit site or, even better, Chapters 1–3 of *Minimalist Data Wrangling with Python*.

This task is due on **Week 2 (12th Jan, Sunday)**. However, ideally, you should complete it by the end of **Week 1**. Hence, start tackling it as early as possible. If we find your first solution incomplete or otherwise incorrect, you will still be able to amend it based on the generous feedback we will give you (allow 3–5 working days). In case of any problems/questions, do not hesitate to attend our online seminars or use the Discussion Board on the unit site.

Submitting after the aforementioned due date might incur a late penalty. The **cut-off date is Week 3 (Friday)**. There will be **no extensions** (this is a Week 1 task, after all) and no solutions will be accepted thereafter. At that time, if your submission is not 100% complete, it will be marked as FAIL, without the possibility of correcting and resubmitting. This task is part of the **hurdle requirements** in this unit. Not submitting the correct version on time results in failing the unit.

All submissions will be checked for plagiarism. You are expected to work independently on your task solutions. Never share/show parts of solutions with/to anyone.

2 Task overview

The main source of income for the Australian Government is income tax, which is collected by the Australian Taxation Office (ATO) and applied to the taxable income of individuals. Suppose your employer pays your salary on a weekly basis. They will deduct the weekly withholding tax from your salary and send it to the ATO. At the end of the financial year, you can lodge a tax return to calculate your tax liability. Often, your employer will have already paid a little too much tax to the ATO, making you eligible for a tax return.

In this exercise, you will be asked to complete the following questions:

1. Generate 10 individuals' weekly salaries using the given distribution.
2. Calculate their yearly income tax based on the Australian resident tax rates for 2023–24.
3. Calculate their weekly withholding tax using the withholding tax formula provided by the ATO.
4. Calculate each individual's tax return.
5. Calculate the employer's weekly before-tax superannuation contribution.
6. Repeat Subtasks 2, 3, 4 to recalculate the income tax, withholding tax, and tax return.

3 Questions

Create a single Jupyter/IPython notebook (see the *Artefacts* section below for all the requirements – read the whole task specification first!), where you perform what follows.

- Q1. Using the given distribution and parameters below, generate a Python list (or array or other types) of weekly before-tax salary of 10 individuals. You can round the results to integers or leave them as decimal numbers.

```
# import numpy as np
weeklySalary = np.random.normal(1431, 527, 10)
```

For the following subtasks, do not use `numpy` nor `pandas`. This is an exercise on base Python.

Q2. For exercise purposes, we make the following assumptions:

- All individuals are Australian residents for tax purposes.
- All individuals are paid weekly, and there are 52 pays in a financial year.
- All individuals claim the tax-free threshold.
- No individuals claim deductions to reduce taxable income.

Using a *for* loop, create (programmatically) a Python list (or array or other types) `incomeTax` such that `incomeTax[i]` gives the annual income tax of the *i*-th individual in the list `weeklySalary` generated in Q1. Please use the tax rates provided in the table below to calculate the income tax.

Taxable income	Tax on this income
\$0 – \$18,200	Nil
\$18,201 – \$45,000	19c for each 1 over \$18,200
\$45,001 – \$120,000	\$5,092 plus 32.5c for each 1 over \$45,000
\$120,001 – \$180,000	\$29,467 plus 37c for each 1 over \$120,000
\$180,001 and over	\$51,667 plus 45c for each 1 over \$180,000

Note that in the tax table, the amount $\$5,092 = 0.19 * (\$45,000 - \$18,200)$ in the third bracket.

As an example, an individual's taxable income for the last financial year was \$130,000. The tax payable would be calculated as

$$0.19 * (45000 - 18200) + 0.325 * (120000 - 45000) + 0.37 * (130000 - 120000) = 33167.$$

Alternatively, it would be calculated as

$$29467 + 0.37 * (130000 - 120000) = 33167,$$

where the amount \$29,467 is the pre-calculated amount for the tax payable in all the previous brackets.

Q3. Each week, the employer will deduct from an employee's weekly salary a withholding tax and pay it to the ATO. The withholding tax is calculated based on the individual's weekly salary using the following mathematical formula:

$$y = a * x - b,$$

where *x* is the weekly salary plus 99 cents, *a* and *b* are coefficients.

Using a *for* loop, create (programmatically) a Python list (or array or other types) `withholdingTax` such that `withholdingTax[i]` gives the weekly withholding tax amount of the *i*-th individual in the list `weeklySalary` generated in Q1. Please use the formula coefficients provided in the table below to calculate the withholding tax. Note that for weekly income less than \$359, the withholding tax is \$0.

Weekly earnings (x) less than \$	a	b
359	–	–
438	0.1900	68.3462
548	0.2900	112.1942
721	0.2100	68.3465
865	0.2190	74.8369
1,282	0.3477	186.2119
2,307	0.3450	182.7504
3,461	0.3900	286.5965

3,461 & over	0.4700	563.5196
--------------	--------	----------

As an example, an individual's annual salary is \$130,000 which corresponds to a weekly salary \$2,500. The withholding tax would be calculated as

$$0.3900 * (2500 + 0.99) - 286.5965 = 688.7896 \approx 689.$$

Q4. Using a *for* loop, create (programmatically) a Python list (or array or other types) `taxReturn` such that `taxReturn[i]` gives the annual eligible tax refund of the *i*-th individual in the list `weeklySalary` generated in Q1. The eligible tax refund should be calculated based on the previously calculated income tax and withholding tax.

As an example, an individual's annual salary is \$130,000. From previous steps, the income tax is \$33,167 and the withholding tax for the entire year is \$689 * 52 = \$35,828, so the tax refundable is \$35,828 - \$33,167 = \$2,661.

Use a *for* loop to print a series of strings like the following. Note that income tax and tax return in the print are calculated on an annual basis. Weekly salary is before tax and weekly income is minus withholding tax.

```
## Person 1 weekly salaryweekly withholding tax $ _weekly income
$ _
income tax $ _tax return $ _.
```

```
## Person 2 weekly salaryweekly withholding tax $ _weekly income
$ _
income tax $ _tax return $ _.
```

```
## Person 3 weekly salaryweekly withholding tax $ _weekly income
$ _
income tax $ _tax return $ _.
```

```
## Person 4 weekly salaryweekly withholding tax $ _weekly income
$ _
income tax $ _tax return $ _.
```

```
## Person 5 weekly salaryweekly withholding tax $ _weekly income
$ _
income tax $ _tax return $ _.
```

```
## Person 6 weekly salaryweekly withholding tax $ _weekly income
$ _
income tax $ _tax return $ _.
```

```
## Person 7 weekly salaryweekly withholding tax $ _weekly income
$ _
income tax $ _tax return $ _.
```

```
## Person 8 weekly salaryweekly withholding tax $ _weekly income
$ _
income tax $ _tax return $ _.
```

```
## Person 9 weekly salaryweekly withholding tax $ _weekly income
$ _
income tax $ _tax return $ _.
```

```
## Person 10 weekly salaryweekly withholding tax $ _weekly income
$ _
income tax $ _tax return $ _.
```

Make sure the formatting is neat and tidy.

Note that your code must work correctly if someone decides to modify these lists: for example, add another person to the list or change their salary amount.

Do not hardcode the values of the income tax, withholding tax, tax return – they must be computed programmatically.

Minimise the printing of unnecessary objects.

Suppose the amount generated in Subtask 1 consists of a base salary and a superannuation that is 11% of the salary. The superannuation is paid by the employer directly to the employee's nominated superannuation account before any tax is deducted. This contribution is known as a before-tax superannuation contribution. In other words, the before-tax superannuation contribution is treated as a deduction from the weekly (or annual) salary and is not subject to normal taxation. While it will be subject to superannuation tax within the superannuation account, we will ignore this aspect to maintain simplicity in this exercise.

As an example, an individual's total employment package is \$130,000, which consists of \$117,117 salary and \$12,883 of superannuation. Suppose the \$12,883 superannuation is contributed to the individual's nominated account as before-tax superannuation contribution, then the individual's taxable income for the financial year is \$117,117. The income tax and weekly withholding tax can then be calculated as illustrated in Subtasks 2 and 3.

- Q5. Using a *for* loop, create (programmatically) a Python list (or array or other types) `superContribution` such that `superContribution[i]` gives the weekly before-tax superannuation contribution of the *i*-th individual in the list `weeklySalary` generated in Q1.
- Q6. By considering the before-tax superannuation contribution as a form of income deduction, repeat Subtasks 2, 3, and 4 to recalculate the individuals' annual income tax, withholding tax, and tax return.

Use a *for* loop to print a series of strings like the following. Note that income tax and tax return in the print are calculated on an annual basis. Weekly salary is before tax, including superannuation. Weekly income is minus weekly withholding tax and superannuation contribution.

```
##      weekly salary $_      weekly superannuation contribution $_
Person weekly withholding tax $_      weekly income $_ income tax
1      $_      tax return $_.
##      weekly salary $_      weekly superannuation contribution $_
Person weekly withholding tax $_      weekly income $_ income tax
2      $_      tax return $_.
##      weekly salary $_      weekly superannuation contribution $_
Person weekly withholding tax $_      weekly income $_ income tax
3      $_      tax return $_.
##      weekly salary $_      weekly superannuation contribution $_
Person weekly withholding tax $_      weekly income $_ income tax
4      $_      tax return $_.
##      weekly salary $_      weekly superannuation contribution $_
Person weekly withholding tax $_      weekly income $_ income tax
5      $_      tax return $_.
##      weekly salary $_      weekly superannuation contribution $_
Person weekly withholding tax $_      weekly income $_ income tax
6      $_      tax return $_.
##      weekly salary $_      weekly superannuation contribution $_
Person weekly withholding tax $_      weekly income $_ income tax
7      $_      tax return $_.
```

```
##      weekly salary $_      weekly superannuation contribution $_
Person weekly withholding tax $_      weekly income $_ income tax
8      $_      tax return $_.
##      weekly salary $_ weekly superannuation contribution $_
Person
9
```

Make sure the formatting is neat and tidy.

4 Artefacts

The solution to the task must be included in a single Jupyter/IPython notebook (an .ipynb file) running against a Python 3 kernel. The use of G**gle Colab is discouraged. Nothing beats a locally-installed version where you have full control over the environment. Do not become dependent on third-party middlemen/distributors. Choose freedom instead.

Make sure that your notebook has a **readable structure**; in particular, that it is divided into sections. Use rich Markdown formatting (text in dedicated Markdown chunks – not just Python comments).

Do not include the questions/tasks from the task specification. Your notebook should read nicely and smoothly – like a report from data analysis that you designed yourself. Make the flow read natural (e.g., *First, let us load the data on... Then, let us determine... etc.*). Imagine it is a piece of work that you would like to show to your manager or clients — you certainly want to make a good impression. Check your spelling and grammar. Also, use formal language.

At the start of the notebook, you need to provide: the **title** of the report (e.g., *Task42:How Much I Love This Unit*), your **name, student number and email address**.

Then, add 1–2 introductory paragraphs (an introduction/abstract – what the task is about).

Before each nontrivial code chunk, briefly **explain** what its purpose is. After each code chunk, **summarise and discuss the obtained results** (in a few sentences).

Conclude the report with 1–2 paragraphs (summary/discussion/possible extensions of the analysis etc.).

Limitations of the ipynb-to-pdf renderer:

Ensure that your report as seen in Olympus is aesthetic. The ipynb-to-pdf renderer is imperfect. We work with what we have. Here are the most common Markdown-related errors.

- Do not include any externally loaded images (via the `! [label] (href)` Markdown command), for they lead to upload errors.
 - Do not input HTML code in Markdown.
 - Make sure you leave one blank line before and after each paragraph and bullet list. Do not use backslashes at the end of the line.
 - Currently, also LaTeX formulae and Markdown tables are not recognised. However, they do not lead to any errors.
-

Checklist:

1. Header, introduction, conclusion (Markdown chunks).

2. Text divided into sections, all major code chunks commented and discussed in your own words (Markdown chunks).
3. Every subtask addressed/solved. In particular, all reference results that are part of the task specification have been reproduced (plots, computed aggregates, etc.).
4. The report is readable and neat. In particular:
 - all code lines are visible in their entirety (they are not too long),
 - code chunks use consecutive numbering (select *Kernel - Restart and Run All* from the Jupyter menu),
 - rich Markdown formatting is used (`# Section Title`, `* bullet list`, `1. enumerated list`, `| table |`, `*italic*`, etc.),
 - the printing of unnecessary/intermediate objects is minimised (focus on reporting the results specifically requested in the task specification).

Submissions which do not *fully* (100%) conform to the task specification *on* the cut-off date will be marked as FAIL.

Good luck!

5 Intended Learning Outcomes

ULO	Is Related?
ULO1 (Data Processing/Wrangling)	YES
ULO2 (Data Discovery/Extraction)	
ULO3 (Requirement Analysis/Data Sources)	
ULO4 (Exploratory Data Analysis)	
ULO5 (Data Privacy and Ethics)	YES